

Technical Note: Automated CRM Client Sync

How it works

A Python script (sync.py) reads the incoming CSV, validates and cleans each row, and upserts records into a local SQLite database (crm.db) in one command. watcher.py monitors a drop folder and triggers the sync automatically whenever a new CSV file arrives, then archives the processed file with a timestamp.

Eight data quality issues in the sample are detected and cleaned: duplicate client IDs (keep the most recently updated record), inconsistent date formats (normalize to YYYY-MM-DD), comma-formatted numbers (strip and parse as numeric), invalid email addresses (stored with email_valid=0 so they cannot be used downstream), missing phone numbers and portfolio values (stored as NULL), a negative portfolio value of -50,000 (NULled out -- all other values in the file are between 640,000 and 2,100,000, so this is clearly a manual entry error; storing it would corrupt any average or sum calculation), and inconsistent UAE phone formatting (normalized to +971 XX XXX XXXX). Original bad values are preserved in sync_issues for audit. Every run also writes to sync_log.txt and a sync_runs table.

Keeping the data accurate and secure over time

The script validates column headers before processing any rows and aborts if something is missing, so a provider format change does not silently corrupt the database. For recurring use I would schedule it via cron and review the sync_issues table weekly for patterns. For security, crm.db should have restricted file permissions, the incoming CSV should be archived after each import rather than left in a shared folder, and credentials should live in environment variables if this ever moves to a server.

What is most likely to go wrong

A renamed column that still has the right field count would pass the header check and load incorrect data into the wrong column. The fix is matching exact column names rather than just checking for missing ones. The other risk is an interrupted run leaving the database in a partial state, but the entire import is already wrapped in a single transaction, so a mid-run crash rolls everything back cleanly.